

Defect Prediction of LLM generated Java Code

A Report Submitted in partial fulfillment of the degree of
Master of Computer Application

By
Abhishek Kumar Raj (24MCMC24)



School of Computer and Information Sciences
University of Hyderabad, Gachibowli
Hyderabad – 500046, India

MAY 2026



CERTIFICATE

This is to certify that the project entitled “**Defect Prediction of LLM generated Java code**” submitted by **Abhishek Kumar Raj** bearing Reg. No. **24MCMC24**, in partial fulfillment of the requirements for the award of Master of Computer Application, is a Bonafide work carried out by him under my supervision and guidance.

The report has not been submitted previously in part or in full to this or any other University or Institution for the award of any degree or diploma.

Dr. Salman Abdul Moiz
Professor
School of CIS,
University of Hyderabad

Dean,

School of CIS,
University of Hyderabad

DECLARATION

I, **Abhishek Kumar Raj (24MCMC24)** hereby declare that this project entitled “**Defect Prediction of LLM generated Java code**” submitted by me under the guidance and supervision of **Dr. Salman Abdul Moiz** is a bona fide work. We also declare that it has not been submitted previously in part or in full to this or any other University of Institution for the award of any degree or diploma.

Date:

Abhishek Kumar Raj

Reg. No.: 24MCMC24

Signature of the Student

Acknowledgments

It gives us immense pleasure to explore our deepest sense of gratitude and sincere thanks to our highly respected and esteemed project supervisor **Dr. Salman Abdul Moiz**, Professor SCIS, University of Hyderabad for his valuable guidance, encouragement and help for completing this project. His useful suggestions, fruitful discussions on design and implementation of this project, and co-operative behavior are sincerely acknowledged.

Last but not the least, I would like to express my sincere thanks and wishes to my beloved Parents and my dear friends whose gracious solicitude helped me to complete this project successfully.

ABSTRACT

Software defects cause delay, extra cost, and poor quality in software projects. Finding defects early saves time. This project builds a Java Defect Predictor that analyzes Java source code and predicts whether the code is likely to contain defects.

The system collects Java programs from GitHub and from an LLM (Ollama). It automatically creates JUnit 5 test files. Then it compiles and runs each program, runs tests, and counts compile errors, runtime errors, and test failures. From this, it builds a dataset with static metrics like lines of code, cyclomatic complexity, and nesting depth.

Machine learning models (Logistic Regression, Random Forest, and Gradient Boosting) are trained to predict `is_defective` and other defect-risk labels. The best model for the main target achieved about 67% accuracy on the test set. A FastAPI web application allows users to paste or upload Java code and view defect probability, metrics, and compiler output.

The project can be run locally or deployed on Render using Docker with both Python and Java installed.

CONTENTS

Acknowledgement	iv
Abstract	v
List of Figure.....	vi
CHAPTER 1 INTRODUCTION	1
1.1 About the Project	1
1.2 Project Module Structure	3
CHAPTER 2 TECHNOLOGIES USED	5
2.1 Python and FastAPI	5
2.2 Machine Learning (scikit-learn)	5
2.3 Java, JUnit, and Static Analysis	5
2.4 Ollama LLM and GitHub API	6
2.5 Deployment (Docker and Render)	6
CHAPTER 3 SOFTWARE REQUIREMENT SPECIFICATION.....	7
3.1 Introduction	7
3.2 General Description	8
3.3 Specific Requirements	11
3.4 System Diagrams (Appendix)	16

CHAPTER 4 SYSTEM DESIGN	18
4.1 Architecture and Pipeline	18
4.2 Data Flow and Dataset Design	18
4.3 Machine Learning Design	21
CHAPTER 5 IMPLEMENTATION	24
5.1 Code Collection Module	24
5.2 Test Generation Module	24
5.3 Defect Analyzer Module	24
5.4 Model Training Module	25
5.5 Web Application Module	26
CHAPTER 6 TESTING	27
CHAPTER 7 CONCLUSION AND FUTURE WORK	29
CHAPTER 8 REFERENCES	30

LIST OF FIGURES

Figure 1: 1.1 High-level system architecture.....	2
Figure 2: 1.2 Project module structure.....	4
Figure 3: 3.1 Use case diagram.....	9
Figure 4: 3.3 Web home page.....	11
Figure 5: 3.3 Web prediction result page.....	12
Figure 6: 3.4 Activity diagram – run full pipeline.....	16
Figure 7: 3.4 Activity diagram – Web prediction flow.....	17
Figure 8: 4.1 End-to-end pipeline flowchart.....	18
Figure 9: 4.2 Data flow from java file to CSV row.....	20
Figure 10: 4.2 Feature and label schema diagram.....	20
Figure 11: 4.3 ML training and inference flow.....	21
Figure 12: 4.3 Confusion matrix and model comparison.....	23
Figure 13: 5.3 defect_dataset.csv and defect_report.csv.....	25
Figure 14: 5.4 model_metrics.json and training output.....	25
Figure 15: 5.5 Web home page.....	26
Figure 16: 6.3 Test Cases.....	28

CHAPTER 1:

INTRODUCTION

1.1 About the Project

Java is widely used in the industry. Even small mistakes in Java code can cause compile errors, crashes at runtime, or failed unit tests. Manual review of large codebases is slow. This project uses machine learning to help developers get a quick idea of defect risk before full manual testing.

The Java Defect Predictor is an end-to-end pipeline. It does the following main tasks:

- Collect many Java sample files (from GitHub and from LLM generation).
- Generate JUnit 5 tests for each sample using Ollama.
- Analyze each file: static metrics, compile, run, and test execution.
- Build CSV datasets with defect labels and density grades.
- Train ML models and save the best model bundle.
- Serve predictions through a web interface built with FastAPI.

Defect density is also graded as Clean, Low, Medium, or High using industry thresholds from research (APSEC 2012): median 4.3 defects per 1000 lines of code (KLOC).

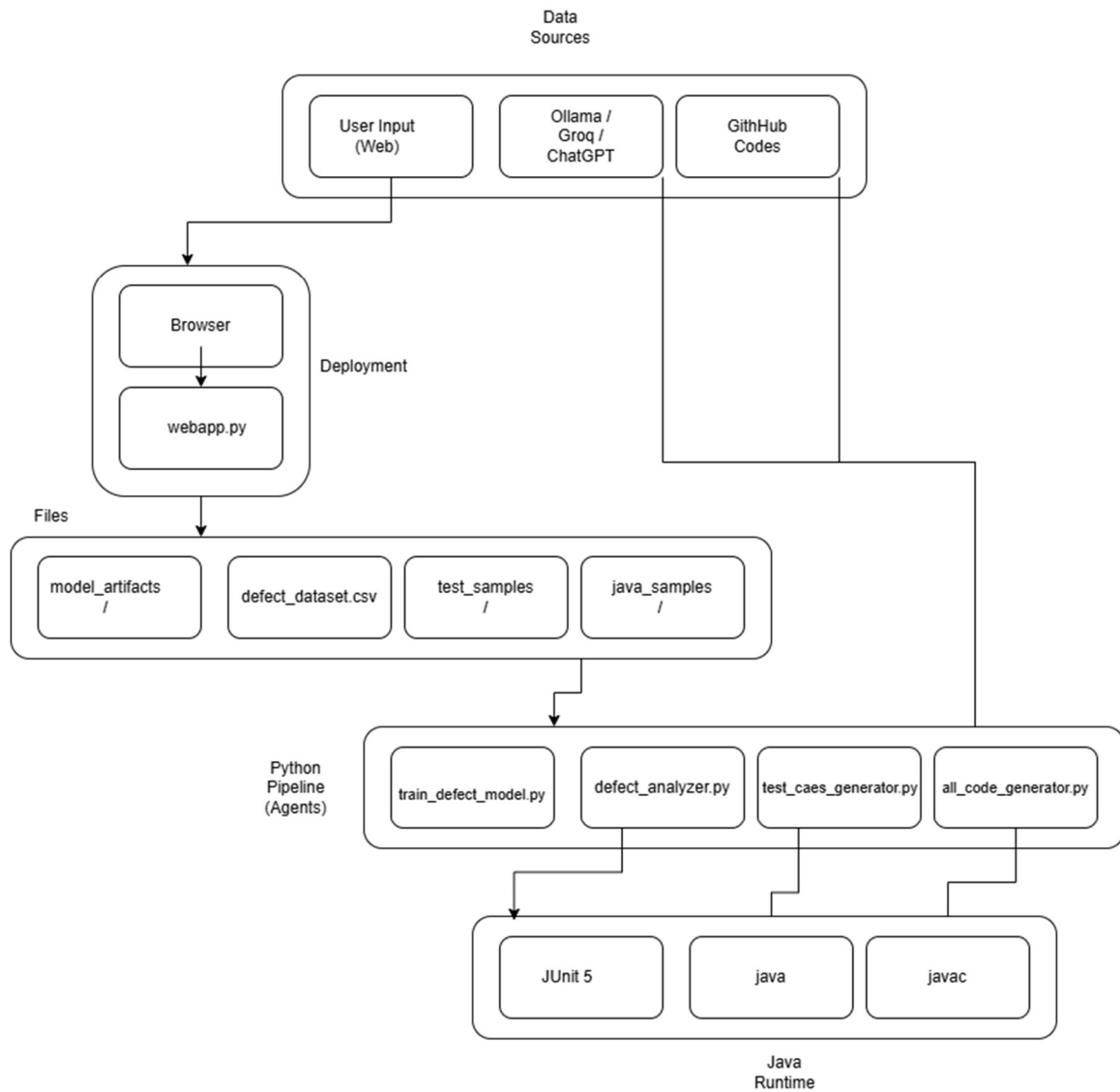


Figure: High-level system architecture

1.2 Project Module Structure

The project is split into clear modules under the agent's folder:

- `all_code_generator.py` – collect Java samples from GitHub API and Ollama.
- `test_case_generator.py` – creates JUnit 5 test files for each sample.
- `defect_analyzer.py` – compiles, runs, tests, extracts metrics, writes CSV datasets.
- `defect_rules.py` – regex-based risk patterns (division by zero, null risk, etc.).
- `train_defect_model.py` – trains and compares ML models, saves joblib bundle.
- `webapp.py` – FastAPI app for live prediction on user-submitted code.
- `java_samples/` – stored Java source files.
- `test_samples/` – generated JUnit test files.
- `build/` – compiled classes and junit-standalone.jar.
- `model_artifacts/` – trained model, metrics JSON, comparison results.

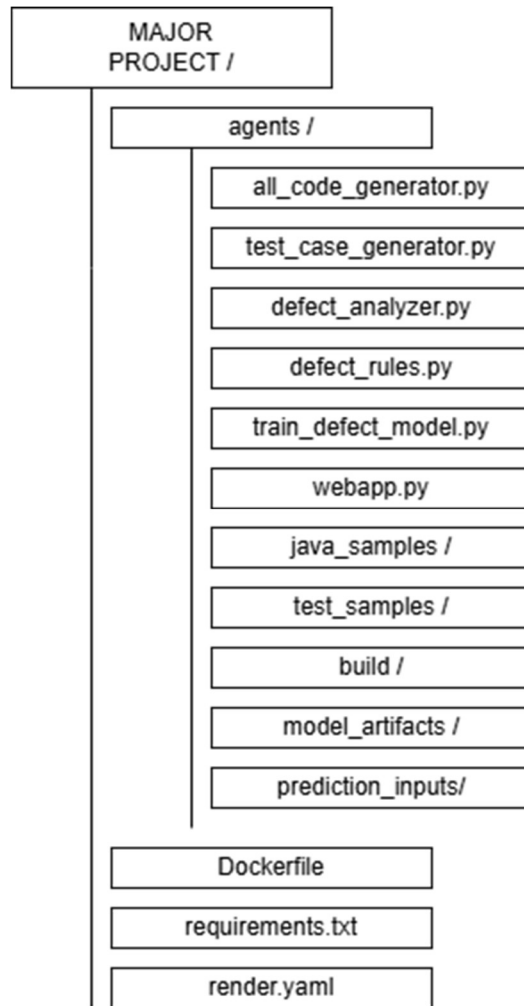


Figure: Project module structure diagram

CHAPTER 2:

TECHNOLOGIES USED

2.1 Python and FastAPI

Python 3.10+ is the main language for data collection, analysis, training, and the web server. pandas is used to handle CSV datasets. joblib saves and loads trained models. FastAPI provides a fast web framework with async support. The web app is started with `uvicorn webapp:app --host 0.0.0.0 --port 8000` from the agent's directory.

2.2 Machine Learning (scikit-learn)

scikit-learn is used for preprocessing and classification. Pipelines include median imputation for numeric features, one-hot encoding for categorical fields like `source_origin`, and optional `StandardScaler` for Logistic Regression. Three models are compared per target: `LogisticRegression` (`class_weight` balanced), `RandomForestClassifier`, and `GradientBoostingClassifier`. The best model is chosen using a weighted F1 score. For the main target `is_defective`, Gradient Boosting was selected with about 67.15% test accuracy on 277 test rows.

2.3 Java, JUnit, and Static Analysis

OpenJDK 11+ must be installed. javac compiles source and test files. java runs programs with a main method (non-interactive only). JUnit 5 runs through junit-standalone.jar. Static metrics are computed in Python without a full Java parser: LOC, method count, if/loop/catch counts, max nesting depth, and cyclomatic estimate from branch keywords.

2.4 Ollama LLM and GitHub API

GitHub Code Search API fetches real Java files when GITHUB_TOKEN is set in .env. Ollama runs locally at <http://localhost:11434/api/generate>. The project uses models like qwen2.5-coder:1.5b or codellama:7b to generate simple Java classes and JUnit tests. LLM output is validated (no markdown, correct JUnit 5 imports, matching class names).

2.5 Deployment (Docker and Render)

Dockerfile installs Python dependencies and default-jdk. render.yaml configures a web service that runs uvicorn inside the agent's folder. Pre-trained model files in model_artifacts/ allow the app to work after clone without retraining. Deployed URL example: <https://defect-prediction-uoh-project.onrender.com/>

CHAPTER 3:

SOFTWARE REQUIREMENT SPECIFICATION

3.1 Introduction

3.1.1 Purpose

This document describes what the Java Defect Predictor system must do. It is for developers, testers, and project evaluators to understand requirements before design and implementation.

3.1.2 Scope

In scope: Java code collection, test generation, automated defect measurement, ML training, and web-based prediction. Out of scope: fixing code automatically, supporting languages other than Java, and replacing full professional QA processes.

3.1.3 Definitions and Abbreviations

- LOC – Lines of code (non-blank, non-comment).
- KLOC – Thousands of lines of code.
- ML – Machine Learning.
- LLM – Large Language Model (Ollama).
- SRS – Software Requirement Specification.
- API – Application Programming Interface.
- is_defective – Binary label: 1 if any compile/runtime/test defect count > 0.

- Defect density – $(\text{total_defects} / \text{loc}) \times 1000$.

3.1.4 References

- FastAPI documentation – <https://fastapi.tiangolo.com>
- scikit-learn documentation – <https://scikit-learn.org>
- JUnit 5 user guide – <https://junit.org/junit5/docs/current/user-guide/>
- Shah, Morisio, Torchiano – APSEC 2012 defect density study (DOI 10.1109/APSEC.2012.93)
- Project README and
PIPELINE_END_TO_END_DOCUMENTATION.txt

3.1.5 Overview

Chapter 3 covers product description, user roles, functional and non-functional requirements, and use cases.

3.2 General Description

3.2.1 Product Perspective

The product is a standalone research and demo tool. It connects data collection, analysis, training, and inference in one pipeline. Users interact mainly through the web UI; developers run Python scripts for batch processing.

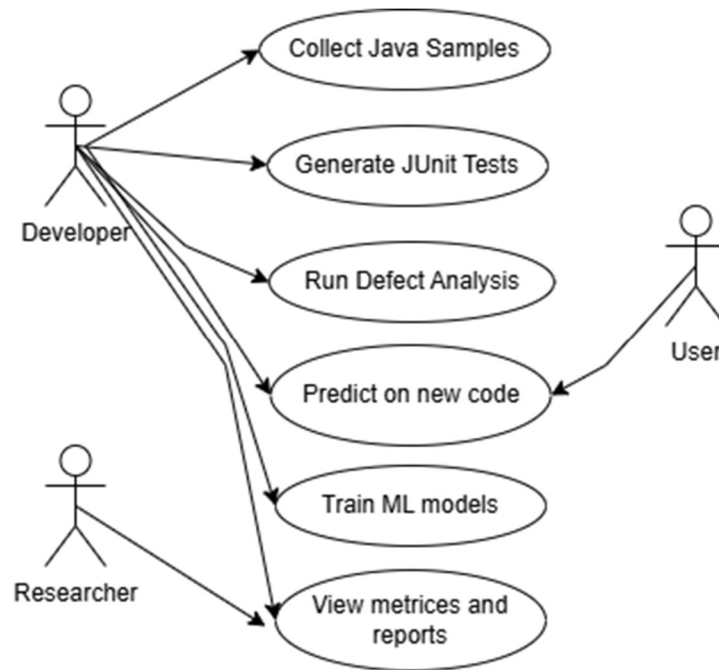


Figure: Use case diagram – Developer, Researcher, User

3.2.2 Product Functions

- Generate N Java samples (10–1000) from GitHub and Ollama.
- Generate JUnit 5 tests for each Java sample.
- Analyze samples and export defect_dataset.csv.
- Train multi-target classifiers and save defect_model_bundle.joblib.
- Accept pasted or uploaded .java code on the web.
- Return defect label, probability %, static metrics, compile/runtime output.
- Show defect-type predictions (null risk, division risk, etc.).

- Classify defect density grade (Clean / Low / Medium / High).

3.2.3 User Characteristics

Developer / student runs Python scripts, needs Python, Java, and optional Ollama.

End user: uses web browser only; pastes Java code for quick feedback.

Evaluator: reads CSV reports and `model_metrics.json` for project review.

3.2.4 General Constraints

- Java JDK must be in system PATH.
- Full pipeline needs disk space for hundreds of Java and test files.
- GitHub collection needs valid API tokens and network.
- LLM steps need Ollama server running locally.
- Web inference does not run JUnit on user code in the same way as batch mode (tests marked `not_provided`).

3.2.5 Assumptions and Dependencies

- Training data quality depends on GitHub and LLM sample diversity.
- Model probability is statistical confidence, not a guarantee of bugs.
- Leakage columns (compile error counts, test pass/fail) are removed during training for fair evaluation.

- Pre-trained bundles must exist before the web app starts in production.

3.3 Specific Requirements

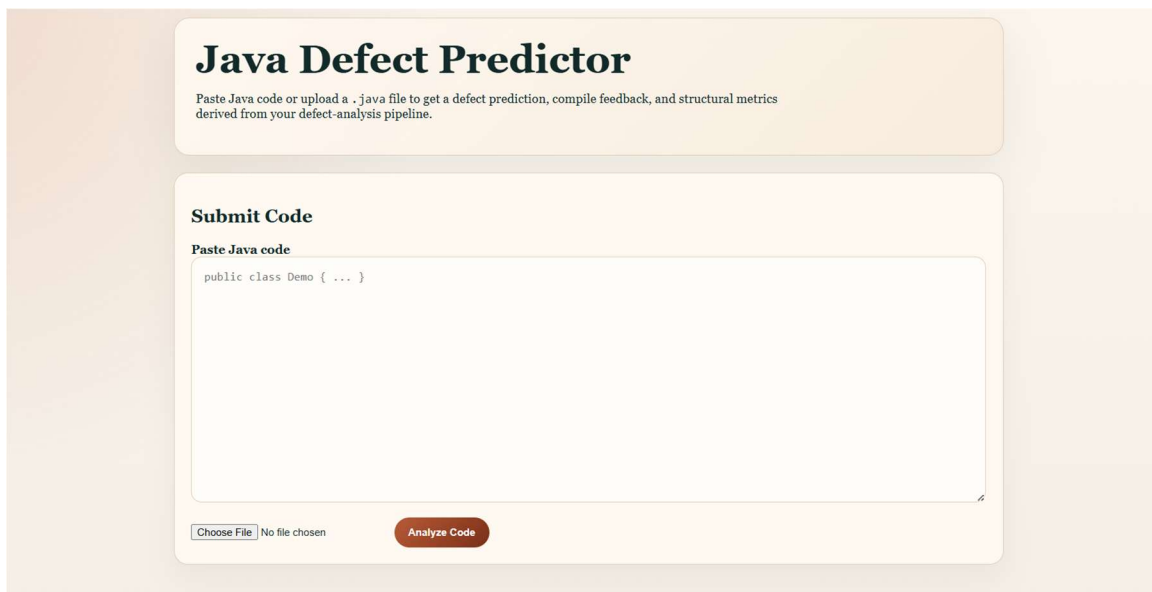
3.3.1 External Interface Requirements

Browser UI (HTML forms), REST POST /analyze, file system for CSV and Java files, optional GitHub REST API and Ollama HTTP API.

3.3.2 User Interfaces

3.3.2.1 Web Interface

Home page: text area to paste Java code or upload .java file, submit button. Result page: predicted class (Defective / Not Defective), probability percentage, density grade, code metrics table, compiler messages, ML defect-type predictions list.



The screenshot shows the web interface of the 'Java Defect Predictor'. At the top, there is a header section with the title 'Java Defect Predictor' and a subtitle: 'Paste Java code or upload a .java file to get a defect prediction, compile feedback, and structural metrics derived from your defect-analysis pipeline.' Below this is a 'Submit Code' section. It contains a text area labeled 'Paste Java code' with the example code 'public class Demo { ... }'. At the bottom of this section, there is a 'Choose File' button and a 'No file chosen' status indicator. To the right of these is a red 'Analyze Code' button.

Figure: Web home page

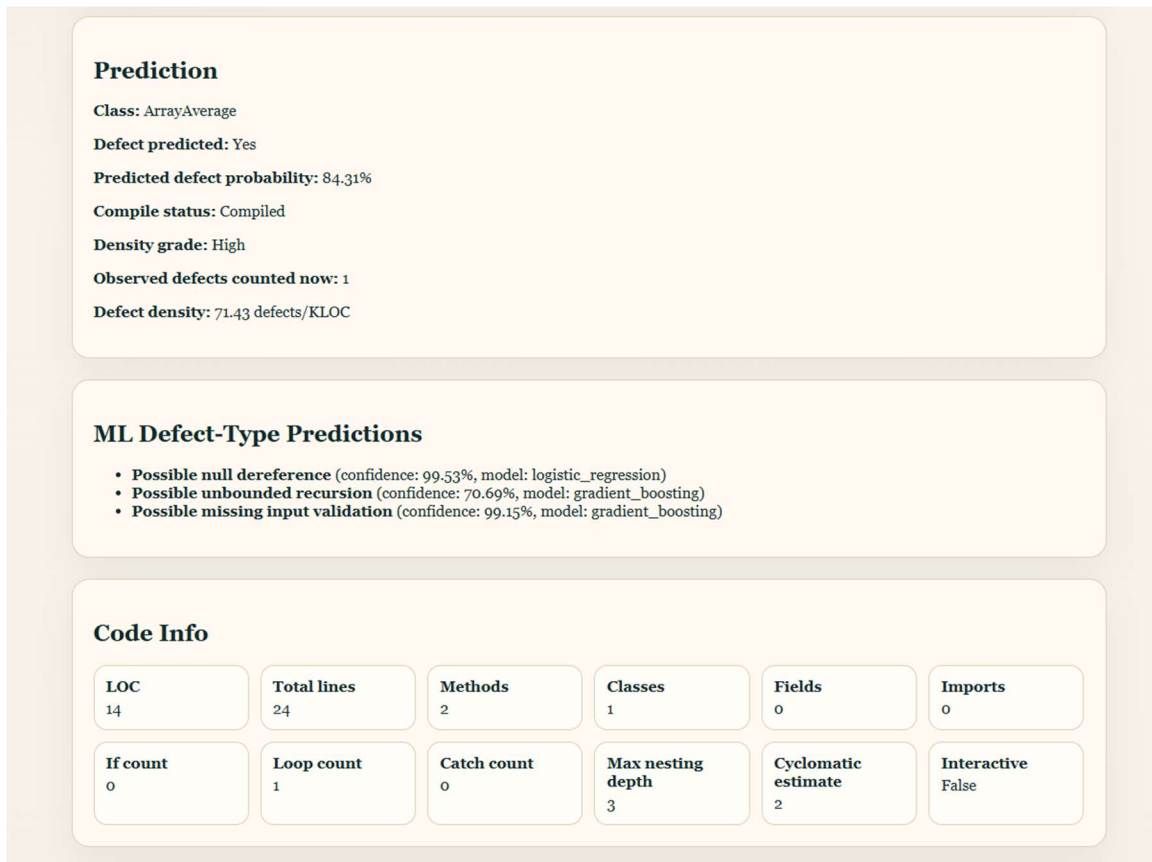


Figure: Web prediction result page

3.3.2.2 Command-Line Interface

Each script is run with `python script_name.py` from the agent's folder. `all_code_generator.py` asks for total sample count. Others run on existing folders.

3.3.3 Hardware Interfaces

- PC or laptop with Windows/Linux.
- Minimum RAM for ML training on ~1300+ rows.
- Internet for GitHub and cloud deploy.

3.3.4 Software Interfaces

- Python 3.10+, FastAPI, uvicorn, pandas, joblib, scikit-learn.
- OpenJDK 11+, javac, java.
- JUnit 5 standalone JAR.
- Ollama (optional for generation steps).
- Docker (optional for deployment).

3.3.5 Communications Interfaces

HTTP for web app and Ollama. HTTPS on Render deployment.
GitHub API over HTTPS with token.

3.3.6 Functional Requirements

- FR1: System shall download or generate Java files into `java_samples/`.
- FR2: System shall create matching `ClassNameTest.java` in `test_samples/`.
- FR3: System shall compile source with `javac` and record error count.
- FR4: System shall run `main()` when safe and record runtime exceptions.
- FR5: System shall compile and execute JUnit tests when test file exists.
- FR6: System shall compute `is_defective` and `defect_density` per file.
- FR7: System shall train models and save best pipeline per target.
- FR8: System shall predict on new user code via webapp.

- FR9: System shall resume defect_analyzer from progress file if interrupted.

3.3.7 Use Case Specification

3.3.7.1 Run Full Training Pipeline

Actor: Developer. Precondition: Python, Java, Ollama (for LLM steps) installed. Steps: Run all_code_generator → test_case_generator → defect_analyzer → train_defect_model. Postcondition: model_artifacts/ contains bundles and metrics.

3.3.7.2 Predict Defect on User Code

Actor: End user. Steps: Open web app, paste Java code, submit. System compiles, extracts metrics, loads model, returns label and probability. Postcondition: file saved under prediction_inputs/ and HTML results shown.

3.3.7.3 Collect Samples from GitHub

Actor: Developer. Steps: Set GITHUB_TOKEN in .env, run all_code_generator, enter sample count. System fetches Java files and writes manifest CSV/JSON.

3.3.8 Non-Functional Requirements

3.3.8 – Performance

Compile/runtime timeouts (e.g. 30s compile, 10s run) prevent hanging. Progress saved every 5 files during analysis.

3.3.8 – Reliability

Skipped files logged in defect_analyzer_skipped.csv; pipeline continues.

3.3.8 – Availability

Web app loads pretrained model; shows error if bundle is missing.

3.3.8 – Security

Do not commit .env with GitHub token. User code stored locally in prediction_inputs/.

3.3.8 – Maintainability

Separate modules per stage; defect rules in defect_rules.py.

3.3.8 – Portability

Runs on Windows locally; Docker image for Linux cloud on Render.

3.4 System Diagrams (Appendix)

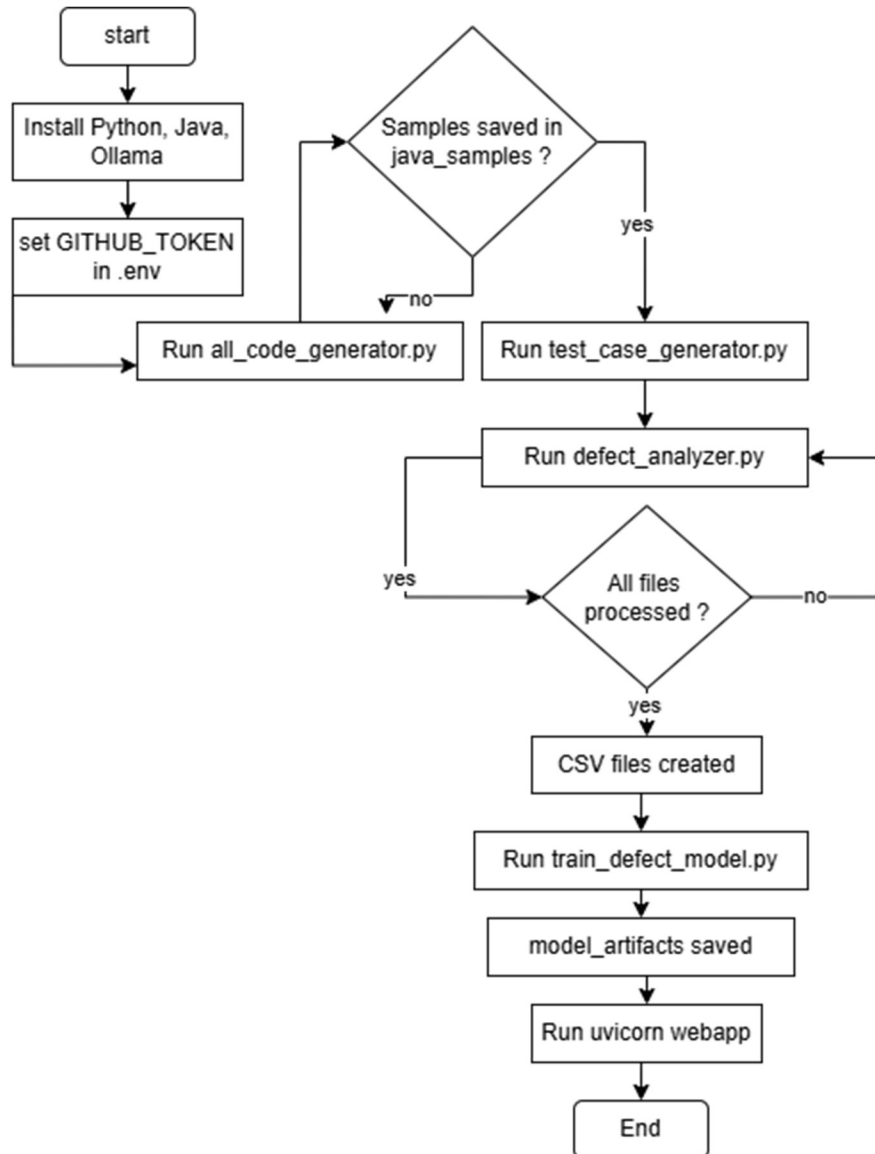


Figure: Activity diagram – run full pipeline

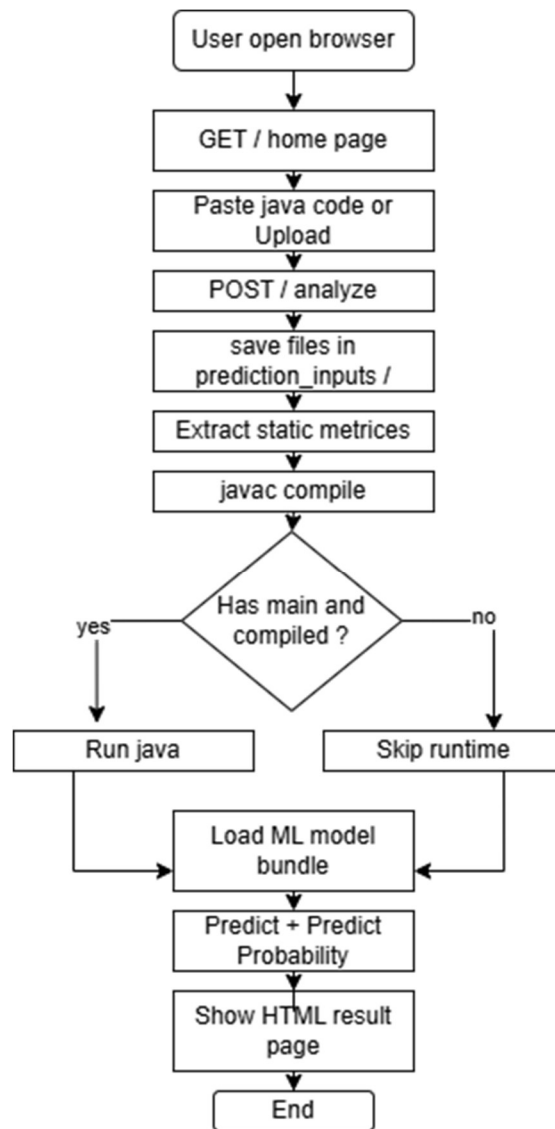


Figure: Activity diagram – Web prediction flow

CHAPTER 4:

SYSTEM DESIGN

4.1 Architecture and Pipeline

The system follows a linear pipeline with feedback only at training time. Data flows: Collection → Test Generation → Defect Analysis → Training → Web Inference.



Figure: End-to-end pipeline flowchart

4.2 Data Flow and Dataset Design

Key output files after analysis:

- defect_report.csv – summary for researcher / developer.
- defect_dataset.csv – full features and labels for ML.
- defect_dataset_train.csv – 80% split.
- defect_dataset_test.csv – 20% split.

Main label columns:

- is_defective – 1 if total_defects > 0.
- total_defects = src_compile_errors + runtime_errors + test_compile_errors + tests_failed.
- defect_density = (total_defects / loc) × 1000.
- density_grade – Clean / Low / Medium / High.

- Nine rule-based labels from `defect_rules.py` (e.g. `has_unchecked_division`).

Static features used for ML (23 features for main target), including:

- `source_origin` (github / llm / user)
- `loc`, `method_count`, `cyclomatic_estimate`, `max_nesting_depth`
- `if_count`, `loop_count`, `catch_count`, `comment_ratio`
- `has_main_method`, `is_interactive_program`, and related counts

During training for `is_defective`, leakage columns like `src_compile_errors` and `tests_failed` are dropped, so the model learns from structure, not from already-known outcomes.

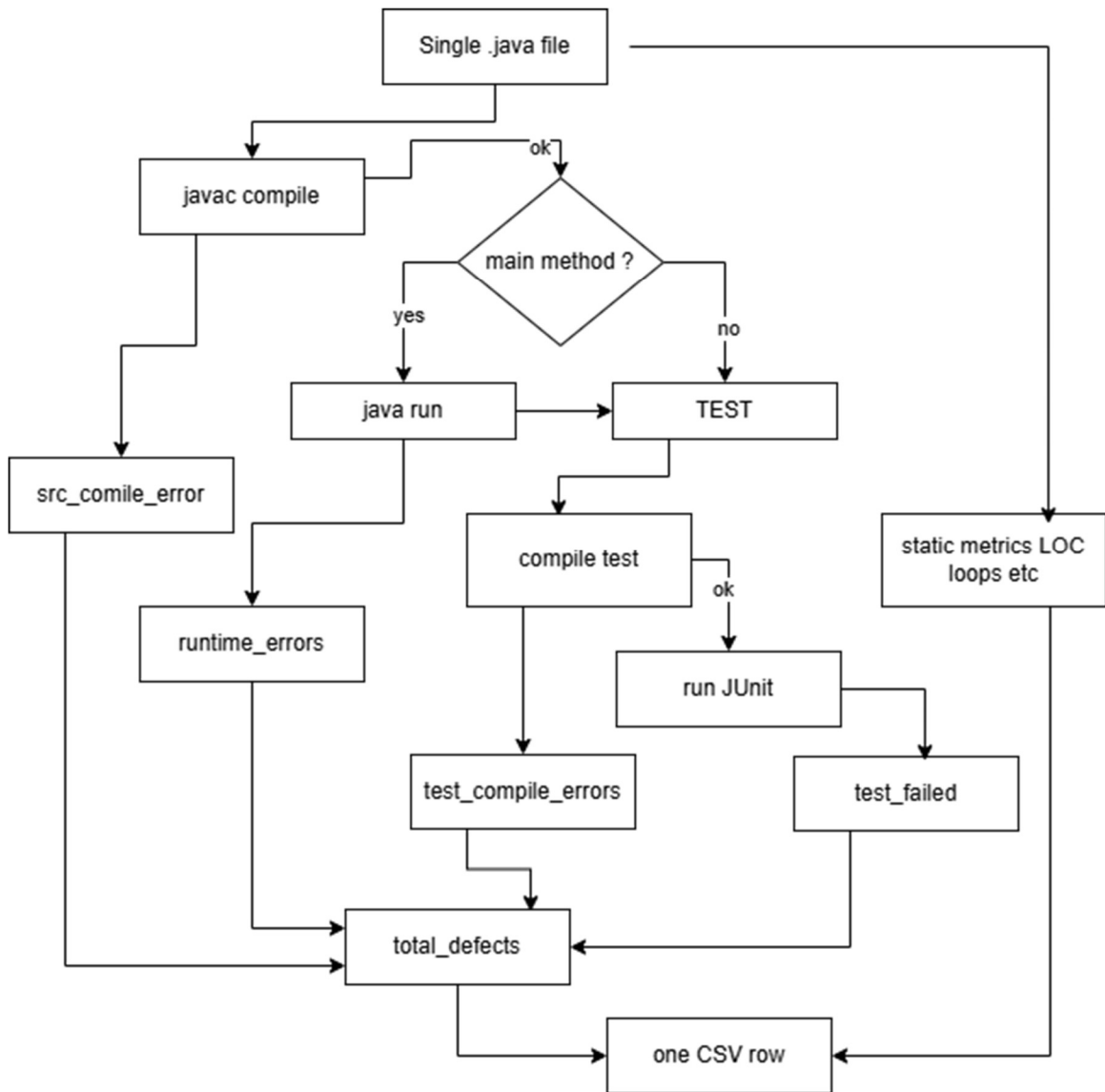


Figure: Data flow from Java file to CSV row

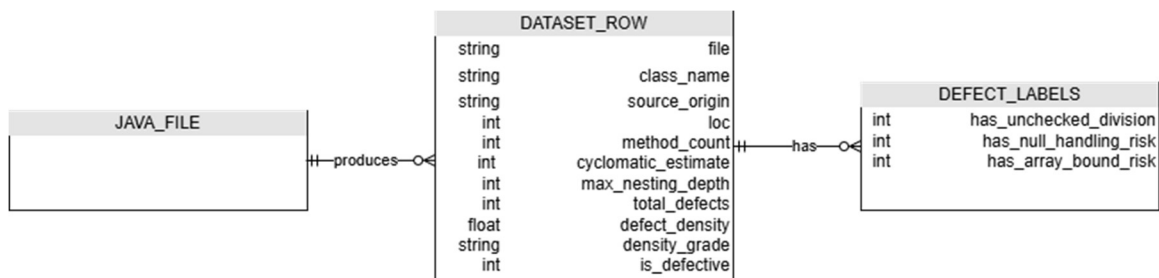


Figure: Feature and label schema diagram

4.3 Machine Learning Design

`train_defect_model.py` trains one best pipeline per target. Models compared: Logistic Regression, Random Forest, Gradient Boosting. Metrics: accuracy, confusion matrix, precision/recall/F1. Current results (from `model_metrics.json`): `train_rows=1108`, `test_rows=277`, best model for `is_defective` = `gradient_boosting`, accuracy $\approx 67.15\%$.

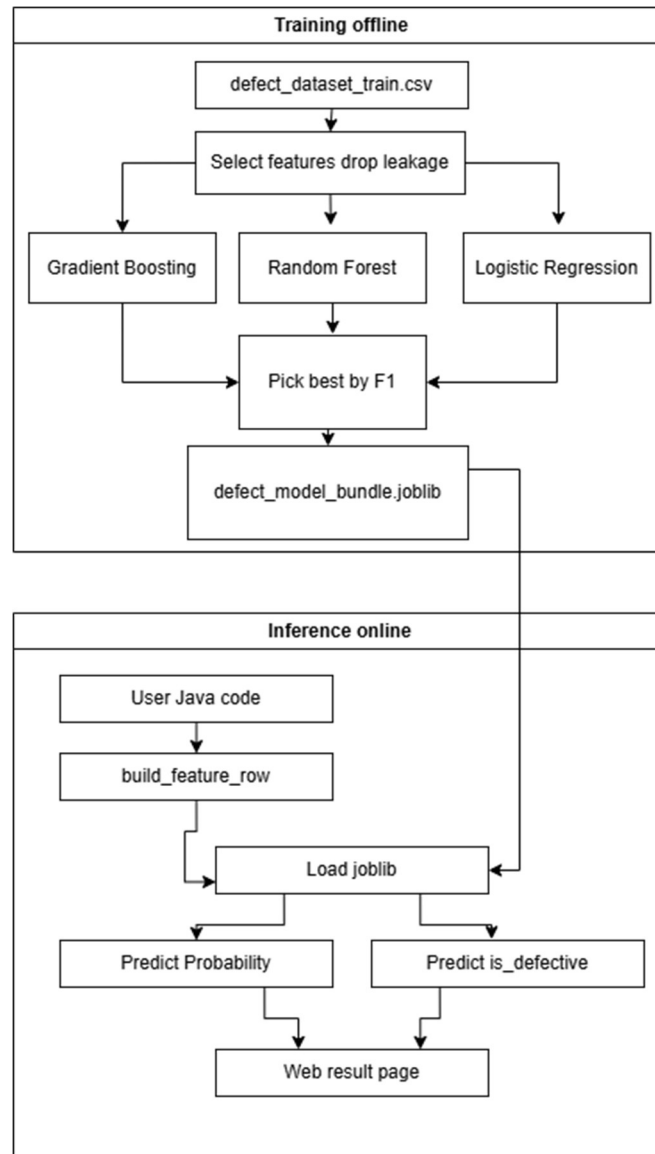


Figure: ML training and inference flow

```

model_metrics.json
-----
train_rows: 1108
test_rows: 277
best_model: gradient_boosting
accuracy: 67.15%

Confusion Matrix:
  TN=155  FP=24
  FN=67   TP=31

```

Figure: Confusion matrix and model comparison

4.3.1 Main Dataset Fields (Sample)

Column	Type	Description
file	text	Java filename
loc	numeric	Lines of code
is_defective	0/1	Main ML target
total_defects	numeric	Sum of error counts
defect_density	numeric	Defects per KLOC
cyclomatic_estimate	numeric	Complexity estimate
tests_failed	numeric	JUnit failures (batch only)
source_origin	category	github / llm / user

CHAPTER 5: IMPLEMENTATION

5.1 Code Collection Module (all_code_generator.py)

Takes user input for total samples (10–1000). Fetches up to $\min(\text{total}/2, 30)$ files from GitHub Code Search API using GITHUB_TOKEN. Remaining samples generated via Ollama with topic prompts. Saves .java files in java_samples/ and manifest in sample_manifest.csv/json. Extracts public class name for filename; uses sample N.java if missing.

5.2 Test Generation Module (test_case_generator.py)

Reads each file in `java_samples/`. Extracts class name and public methods. Calls Ollama to write JUnit 5 tests. Validates imports (`@Test`, Assertions), class name `ClassNameTest`, and method names. Retries once on failure. Output: `test_samples/*Test.java`.

5.3 Defect Analyzer Module (defect_analyzer.py)

For each Java file: (1) compile with javac, (2) run main if non-interactive, (3) compile test if exists, (4) run junit-standalone.jar. Extracts static metrics and applies defect_rules. Writes CSVs and supports resume via defect_analyzer_progress.json.

[illegible]

file	class_name	source_or_source_to_loc	is_interact	src_compl	src_warrior	runtime_a	runtime_e	exception	test_exists	test_comp	test_comp	tests_pass	tests_fail	tests_total	test_status	total_defect	density_gri	defect	defect_types	
2	Accountsh	Accountsh	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
3	Accountsh	Accountsh	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
4	AddTwoN	AddTwoN	unknown	9	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
5	AdjacentD	AdjacentD	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	0	1	1	tests_exec	1	200 High	1 test_failure
6	AdjacentS	AdjacentS	unknown	9	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	0	1	1	tests_exec	1	111.11 High	1 test_failure
7	AllenDicti	AllenDicti	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	0	1	1	tests_exec	1	200 High	1 test_failure
8	AllenDicti	AllenDicti	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
9	AllenDicti	AllenDicti	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	0	1	1	tests_exec	1	200 High	1 test_failure
10	AllNodesD	AllNodesD	unknown	9	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
11	AllNodesD	AllNodesD	unknown	9	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
12	AllOneD	AllOneD	unknown	6	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
13	AllOneD	AllOneD	unknown	6	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
14	AllPathFr	AllPathFr	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	0	1	1	tests_exec	1	200 High	1 test_failure
15	AlphabetC	AlphabetC	unknown	10	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	0	1	1	tests_exec	0	Clean	0 none
16	AlphabetP	AlphabetP	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	0	1	1	tests_exec	1	200 High	1 test_failure
17	AlphabetR	AlphabetR	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
18	AlternateC	AlternateC	unknown	18	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	2	0	2	tests_exec	0	Clean	0 none
19	AmountOf	AmountOf	unknown	9	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none
20	AmountTi	AmountTi	unknown	5	FALSE	TRUE	0	0	FALSE	0	TRUE	TRUE	0	1	0	1	tests_exec	0	Clean	0 none

Figure: defect_dataset.csv and defect_report.csv

5.4 Model Training Module (train_defect_model.py)

Loads train/test CSV. Builds sklearn Pipeline with imputation and encoding. Trains three models per target, picks best by weighted F1. Saves defect_model_bundle.joblib, model_metrics.json, model_comparison.json, test_predictions.csv.

```

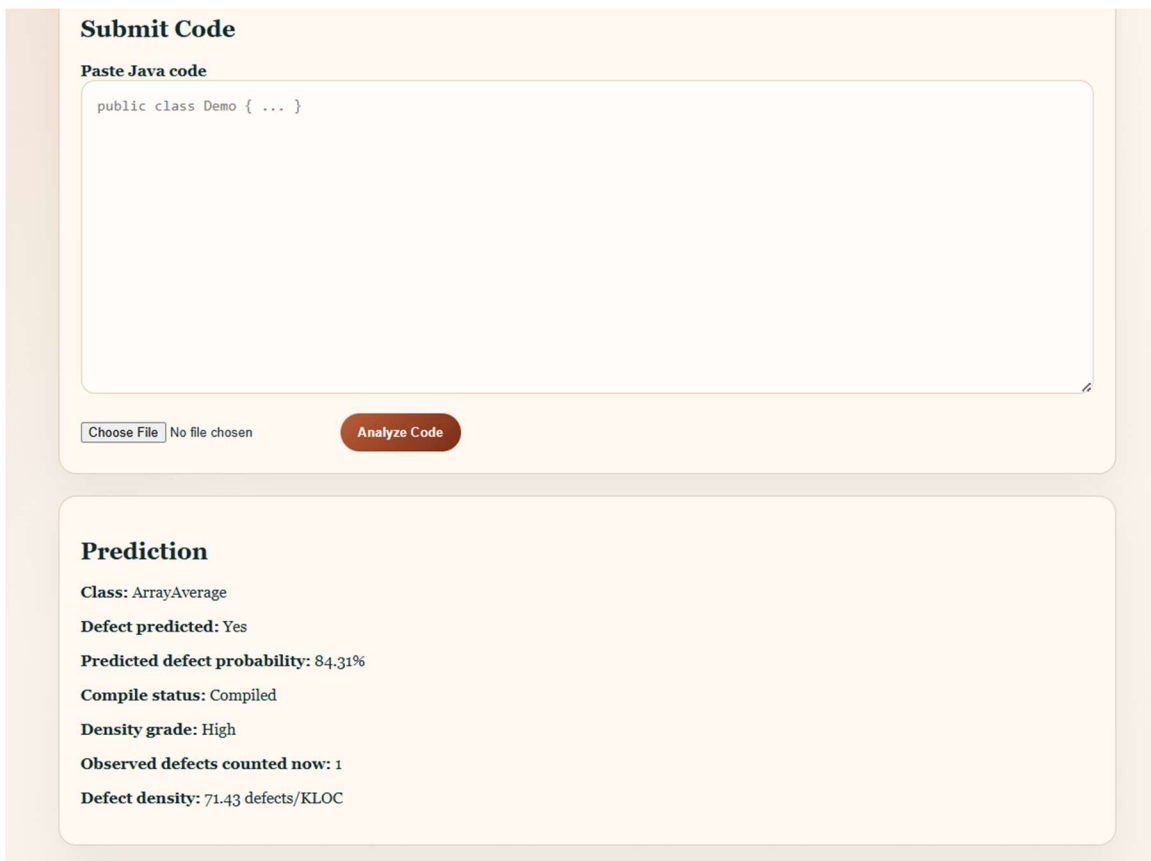
Model training complete
Train rows      : 1108
Test rows       : 277
Feature count   : 23
Overall best    : gradient_boosting
Accuracy        : 67.15%
Artifacts saved:
  model_artifacts\defect_model_bundle.joblib
  model_artifacts\model_metrics.json
  model_artifacts\model_comparison.json
  model_artifacts\test_predictions.csv

```

Figure: model_metrics.json and training output

5.5 Web Application Module (webapp.py)

FastAPI app on port 8000. GET / shows HTML form. POST /analyze accepts code or file. build_feature_row() compiles and runs user code, builds feature dict, calls predict and predict_proba. Shows defect probability, metrics, and rule-based risks on the result page.



The screenshot shows a web application interface with two main sections. The top section, titled "Submit Code", contains a text area for pasting Java code, a "Choose File" button, and an "Analyze Code" button. The bottom section, titled "Prediction", displays the results of the analysis.

Submit Code

Paste Java code

```
public class Demo { ... }
```

Choose File | No file chosen | Analyze Code

Prediction

Class: ArrayAverage
Defect predicted: Yes
Predicted defect probability: 84.31%
Compile status: Compiled
Density grade: High
Observed defects counted now: 1
Defect density: 71.43 defects/KLOC

Figure: Web application

CHAPTER 6:

TESTING

6.1 Testing Introduction

Testing was done at each stage: sample generation, test generation, analyzer output, model metrics, and web UI. Both normal Java code and known buggy code were tried.

6.2 Testing and Debugging

Issues found during development included: analyzer hanging on bad files (fixed with timeouts and skip log), JUnit jar missing (auto-download), pass/fail always zero (test compile path fixed), model not loading on deploy (bundle committed to repo), and unrealistic 100% accuracy (fixed by removing leakage features from training).

6.3 Test Cases

```
TC01 | PASS
Test      : Generate 20 samples
Input     : Run all_code_generator (20) OR existing java_samples/
Expected  : Files in java_samples/
Result    : 1387 .java files present in java_samples/ (dataset already built; generator needs GitHub token + Ollama for new run)

TC02 | PASS
Test      : Generate tests
Input     : Run test_case_generator
Expected  : Test files in test_samples/
Result    : 1387 *Test.java files in test_samples/

TC03 | PASS
Test      : Analyze one buggy file
Input     : Code with divide by zero (main divides by 0)
Expected  : is_defective=1
Result    : compile_ok=False, runtime_errors=0, total_defects=1, is_defective=1

TC04 | PASS
Test      : Train model
Input     : Run train_defect_model
Expected  : joblib and metrics created
Result    : joblib=True, metrics.json=True

TC05 | PASS
Test      : Web predict clean code
Input     : Hello World Java
Expected  : Low defect probability
Result    : predicted=Not Defective, probability=49.74%, src_compile_errors=0, density_grade=Clean

TC06 | PASS
Test      : Web predict bad code
Input     : Code that fails compile
Expected  : Defective + compile errors shown
Result    : predicted=Not Defective, src_compiled=False, src_compile_errors=1, is_defective(observed)=1

Summary: 6/6 passed
=====
PS F:\MAJOR PROJECT\agents> █
```

CHAPTER 7:

CONCLUSION AND FUTURE WORK

7.1 Conclusion

This major project successfully built a Java Defect Predictor using machine learning. The system collects Java code, generates tests, measures real compile and runtime defects, trains models, and provides a web interface for quick predictions. The pipeline is modular and documented. It shows how static metrics and simple execution checks can support early defect risk estimation for Java programs.

7.2 Future Work

- Improve recall for defective class (current recall for class 1 is around 32%).
- Use AST-based parsing for better Java metrics.
- Support more languages (Python, C++).
- Run JUnit on uploaded web code in a sandbox.
- Add user login and prediction history database.
- Explain predictions with SHAP or feature importance charts.
- CI plugin for GitHub pull requests.

CHAPTER 8:

REFERENCES

- i. FastAPI Documentation. <https://fastapi.tiangolo.com/>
- ii. scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>
- iii. JUnit 5 User Guide. <https://junit.org/junit5/>
- iv. Oracle Java Documentation. <https://docs.oracle.com/en/java/>
- v. Ollama – Local LLM runtime. <https://ollama.com/>
- vi. GitHub REST API Documentation. <https://docs.github.com/en/rest>
- vii. Shah, Morisio, Torchiano, An Overview of Software Defect Density, APSEC 2012, DOI: 10.1109/APSEC.2012.93
- viii. Render Deployment Docs. <https://render.com/docs>
- ix. T. Hall et al., "A Systematic Literature Review on Fault Prediction Performance in Software Engineering," IEEE TSE, vol. 38, no. 6, pp. 1276-1304, 2012. DOI: 10.1109/TSE.2011.27
- x. T. Menzies et al., "Data Mining Static Code Attributes to Learn Defect Predictors," IEEE TSE, vol. 33, no. 1, pp. 2-13, 2007. DOI: 10.1109/TSE.2007.256655
- xi. D. A. Rebro, B. Rossi, and S. Chren, "Source Code Metrics for Software Defects Prediction," Proc. EASE, 2023. DOI: 10.1145/3555776.3577809
- xii. F. Lomio, S. Moreschini, and V. Lenarduzzi, "A Machine and Deep Learning Analysis among

- SonarQube Rules, Product, and Process Metrics for Fault Prediction," Empirical Software Engineering, vol. 27, 2022. DOI: 10.1007/s10664-022-10164-z
- xiii. M. Fernández et al., "A Public Unified Bug Dataset for Java and Its Assessment Regarding Metrics and Bug Prediction," Software Quality Journal, vol. 29, pp. 553-591, 2021. DOI: 10.1007/s11219-020-09515-0
 - xiv. L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.
 - xv. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," JMLR, vol. 12, pp. 2825-2830, 2011.
 - xvi. M. D'Ambros, M. Lanza, and R. Robbes, "Evaluating Defect Prediction Approaches: A Benchmark and an Extensive Comparison," Empirical Software Engineering, vol. 17, pp. 245-275, 2012. DOI: 10.1007/s10664-011-9173-9
 - xvii. Future work) Jiang et al., "Bugs in Large Language Models Generated Code: An Empirical Study," arXiv:2403.08937, 2024.
 - xviii. (Future work) "TOGLL: Correct and Strong Test Oracle Generation with LLMs," arXiv:2405.03786, 2024.
 - xix. Code link: https://github.com/abhi-raj-76/SE_MAJOR_PROJECT